Durham University

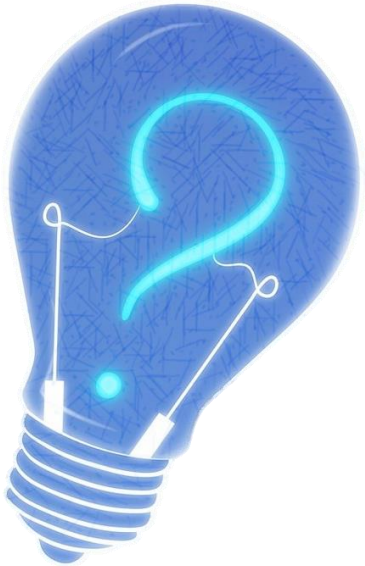
Java Collections
Part 2

Mladen Ivkovic

COMP2221 Programming Paradigms – Object-oriented Programming

What Will You Learn in this Lecture?

- Java Collections Performance Examples
- Best Practices with Java Collections
- Pitfalls with Java Collections, Objects, and References



Durham University

In the Last Lecture...

Recap: Java Collections Categories

The Java Collections Framework is **divided into four main categories**:

1. **`List`:** Ordered collection; allows duplicates.
2. **`Set`:** Unique elements, no duplicates.
3. **`Map`:** Key-value pairs, no duplicate keys.
4. **`Queue`:** FIFO structure, used in task scheduling.

Recap: `LinkedList` vs `ArrayList`

`LinkedList`	`ArrayList`
Slow random access of elements: Needs to traverse all nodes/elements until you reach an index	**Fast random access of elements:** Index specifies location in memory
Quick insertion/removal of elements: Only needs to adapt references to previous/next node/element	**Slow insertion/removal of elements:** Needs to shift around memory for each of those operations

Durham University

Java Collections Performance – `ArrayList` vs `LinkedList` Random Access

```java
import java.util.LinkedList;
import java.util.ArrayList;

public class Collections1 {
  public static void main(String[] args) {

    // experiment sizes
    int n_size[] = {100, 1000, 100000, 1000000};

    // run several experiments of different sizes:
    for (int n : n_size) {

      // make a new list
      ArrayList<Integer> al = new ArrayList<Integer>();
      LinkedList<Integer> ll = new LinkedList<Integer>();

      // add elements with initial values
      for (int i = 0; i < n; i++) {
        al.add(i);
        ll.add(i);
      }
```

```java
long astart = System.currentTimeMillis(); // start time
for (int i = 0; i < n; i++) { // loop over full list
  int r = (int) (Math.random() * n); // get random index
  int  x = al.get(i) + 2 * al.get(r); // random calc.
  al.set(i, x); // store the result
}
long astop = System.currentTimeMillis(); // end time

// same for LinkedList
for (int i = 0; i < n; i++) {
  int r = (int) (Math.random() * n);
  int  x = ll.get(i) + 2 * ll.get(r);
  ll.set(i, x);
}
long lstop = System.currentTimeMillis();

System.out.print("n=" + n + ", ");
System.out.print("ArrayList time [ms]=" +
  (astop - astart) + ", ");
System.out.println("LinkedList time [ms]=" +
  (lstop - astop));
}}}
```

Java Collections Performance – `ArrayList` vs `LinkedList` Random Access

```
import java.util.LinkedList;
import java.util.ArrayList;

public class Collections1 {
  public static void main(String[] args) {

    // experiment sizes
    int n_size[] = {100, 1000, 100000, 1000000};

    // run
    for (i

      // m
      Arra
      Link

      // add elements with initial values
      for (int i = 0; i < n; i++) {
        al.add(i);
        ll.add(i);
      }
```

```
long astart = System.currentTimeMillis(); // start time
for (int i = 0; i < n; i++) { // loop over full list
  int r = (int) (Math.random() * n); // get random index
  int  x = al.get(i) + 2 * al.get(r); // random calc.
  al.set(i, x); // store the result
}
long astop = System.currentTimeMillis(); // end time
```

Output: (indents added manually for clarity)

```
n=100,     ArrayList time [ms]=1,   LinkedList time [ms]=0
n=1000,    ArrayList time [ms]=1,   LinkedList time [ms]=1
n=100000,  ArrayList time [ms]=8,   LinkedList time [ms]=3268
n=1000000, ArrayList time [ms]=148, LinkedList time [ms]=1024273
```

```
System.out.print("n=" + n + ", ");
System.out.print("ArrayList time [ms]=" +
  (astop - astart) + ", ");
System.out.println("LinkedList time [ms]=" +
  (lstop - astop));
}}}
```

Java Collections Performance – `ArrayList` vs `LinkedList` Insertion/Removal at Start/End

```java
import java.util.LinkedList;
import java.util.ArrayList;

public class Collections1 {
  public static void main(String[] args) {
    // experiment sizes
    int n_size[] = {100, 1000, 100000, 1000000};
    // how many times we insert/remove an element
    int n_insertions = 10000;

    // run several experiments of different sizes:
    for (int n : n_size) {

      // make a new list
      ArrayList<Integer> al = new ArrayList<Integer>();
      LinkedList<Integer> ll = new LinkedList<Integer>();

      // add elements with initial values
      for (int i = 0; i < n; i++) {
        al.add(i);
        ll.add(i);
      }-
```

```java
long astart = System.currentTimeMillis(); // start time
for (int ins = 0; ins < n_insertions; ins++) {
  al.add(ins);   // add a new element at the end
  al.remove(0);  // delete first element
}
long astop = System.currentTimeMillis(); // end time

// same for LinkedList
for (int ins = 0; ins < n_insertions; ins++) {
  ll.add(ins);
  ll.remove(0);
}
long lstop = System.currentTimeMillis();

System.out.print("n=" + n + ", ");
System.out.print("ArrayList time [ms]=" +
  (astop - astart) + ", ");
System.out.println("LinkedList time [ms]=" +
  (lstop - astop));
}
}
}
```

Java Collections Performance – `ArrayList` vs `LinkedList` Insertion/Removal at Start/End

```
import java.util.LinkedList;
import java.util.ArrayList;

public class Collections1 {
  public static void main(String[] args) {
    // experiment sizes
    int n_size[] = {100, 1000, 100000, 1000000};
    // how man
    int n_inse

    // run sev
    for (int n

      // make
      ArrayList<Integer> al = new ArrayList<Integer>();
      LinkedList<Integer> ll = new LinkedList<Integer>();

      // add elements with initial values
      for (int i = 0; i < n; i++) {
        al.add(i);
        ll.add(i);
      }-
```

```
long astart = System.currentTimeMillis(); // start time
for (int ins = 0; ins < n_insertions; ins++) {
  al.add(ins);   // add a new element at the end
  al.remove(0);  // delete first element
}
long astop = System.currentTimeMillis(); // end time

                                          ) {

System.out.print("n=" + n + ", ");
System.out.print("ArrayList time [ms]=" +
  (astop - astart) + ", ");
System.out.println("LinkedList time [ms]=" +
  (lstop - astop));
}
}
}
```

Output: (indents added manually for clarity)

```
n=100,     ArrayList time [ms]=1,   LinkedList time [ms]=2
n=1000,    ArrayList time [ms]=1,   LinkedList time [ms]=1
n=100000,  ArrayList time [ms]=59,  LinkedList time [ms]=1
n=1000000, ArrayList time [ms]=889, LinkedList time [ms]=0
```

Java Collections Performance – `ArrayList` vs `LinkedList` Insertion/Removal at Random Positions

```java
import java.util.LinkedList;
import java.util.ArrayList;

public class Collections1 {
  public static void main(String[] args) {
    // experiment sizes
    int n_size[] = {100, 1000, 100000, 1000000};
    // how many times we insert/remove an element
    int n_insertions = 1000;

    // run several experiments of different sizes:
    for (int n : n_size) {

      // make a new list
      ArrayList<Integer> al = new ArrayList<Integer>();
      LinkedList<Integer> ll = new LinkedList<Integer>();

      // add elements with initial values
      for (int i = 0; i < n; i++) {
        al.add(i);
        ll.add(i);
      }
```

```java
long astart = System.currentTimeMillis(); // start time
for (int ins = 0; ins < n_insertions; ins++) {
  int r = (int) (Math.random() * n); // get random index
  al.add(r, ins);  // insert some value at index r
  r = (int) (Math.random() * n);  // get new random index
  al.remove(r); // delete value at random index
}
long astop = System.currentTimeMillis(); // end time

// same for LinkedList
for (int ins = 0; ins < n_insertions; ins++) {
  int r = (int) (Math.random() * n); ll.add(r, ins);
  r = (int) (Math.random() * n); ll.remove(r);
}
long lstop = System.currentTimeMillis();

System.out.print("n=" + n + ", ");
System.out.print("ArrayList time [ms]=" +
  (astop - astart) + ", ");
System.out.println("LinkedList time [ms]=" +
  (lstop - astop));
}}}
```

Java Collections Performance – `ArrayList` vs `LinkedList` Insertion/Removal at Random Positions

```
import java.util.LinkedList;
import java.util.ArrayList;

public class Collections1 {
  public static void main(String[] args) {
    // experiment sizes
    int n_size[] = {100, 1000, 100000, 1000000};
    // how many times we insert/remove an element
    int n_inse

    // run sev
    for (int n

      // make
      ArrayLis
      LinkedList<Integer> ll = new LinkedList<Integer>();

      // add elements with initial values
      for (int i = 0; i < n; i++) {
        al.add(i);
        ll.add(i);
      }
```

```
long astart = System.currentTimeMillis(); // start time
for (int ins = 0; ins < n_insertions; ins++) {
  int r = (int) (Math.random() * n); // get random index
  al.add(r, ins);  // insert some value at index r
  r = (int) (Math.random() * n);  // get new random index
  al.remove(r); // delete value at random index
}
long astop = System.currentTimeMillis(); // end time
```

Output: (indents added manually for clarity)

```
n=100,     ArrayList time [ms]=1,  LinkedList time [ms]=1
n=1000,    ArrayList time [ms]=0,  LinkedList time [ms]=2
n=100000,  ArrayList time [ms]=6,  LinkedList time [ms]=111
n=1000000, ArrayList time [ms]=97, LinkedList time [ms]=2655
```

```
System.out.print("n=" + n + ", ");
System.out.print("ArrayList time [ms]=" +
  (astop - astart) + ", ");
System.out.println("LinkedList time [ms]=" +
  (lstop - astop));
}}}
```

Java Collections Performance – ArrayList vs LinkedList vs HashSet

```
import java.util.LinkedList;
import java.util.ArrayList;
import java.util.HashSet;

public class Collections {
 public static void main(String[] args) {
  int n_size[] = {1000, 50000, 1000000, 10000000};
  int n_elements = 100; // nr unique elements to permit

  for (int n : n_size) {
   ArrayList<Integer> al = new ArrayList<Integer>();
   LinkedList<Integer> ll = new LinkedList<Integer>();
   HashSet<Integer> hs = new HashSet<Integer>();

   long astart = System.currentTimeMillis(); // start time
   for (int i = 0; i < n; i++) {
    // get random value < n_elements
    int r = (int) (Math.random() * n_elements);
    // add if not in list already
    if (! al.contains(r)) { al.add(r); }
   }
   long astop = System.currentTimeMillis(); // end time

   // same for LinkedList
   for (int i = 0; i < n; i++) {
     int r = (int) (Math.random() * n_elements);
     if (! ll.contains(r)) { ll.add(r); }
   }
   long lstop = System.currentTimeMillis();

   // same for HashSet
   for (int i = 0; i < n; i++) {
     int r = (int) (Math.random() * n_elements);
     hs.add(r);
   }
   long hstop = System.currentTimeMillis();

   System.out.print("n=" + n + ", ");
   System.out.print("ArrayList time [ms]=" +
     (astop - astart) + ", ");
   System.out.print("LinkedList time [ms]=" +
     (lstop - astop) + ", ");
   System.out.println("HashSet time [ms]=" +
     (hstop - lstop));
}}}
```

Java Collections Performance – `ArrayList` vs `LinkedList` vs `HashSet`

```
import java.util.LinkedList;
import java.util.ArrayList;
import java.util.HashSet;

public class Collections {
 public static void main(String[] args) {
  int n_size[] = {1000, 50000, 1000000, 10000000};
  for (int i = 0; i < n; i++) {
   // get random value < n_elements
   int r = (int) (Math.random() * n_elements);
   // add if not in list already
   if (! al.contains(r)) { al.add(r); }
  }
  long astop = System.currentTimeMillis(); // end time
  // same for LinkedList
  for (int i = 0; i < n; i++) {
    int r = (int) (Math.random() * n_elements);
    if (! ll.contains(r)) { ll.add(r); }
  }
  long lstop = System.currentTimeMillis();
  System.out.print("ArrayList time [ms]=" +
    (astop - astart) + ", ");
  System.out.print("LinkedList time [ms]=" +
    (lstop - astop) + ", ");
  System.out.println("HashSet time [ms]=" +
    (hstop - lstop));
}}}
```

Output: (indents added manually for clarity)

```
n=1000,     ArrayList time [ms]=1,   LinkedList time [ms]=1,    HashSet time [ms]=0
n=50000,    ArrayList time [ms]=6,   LinkedList time [ms]=8,    HashSet time [ms]=2
n=1000000,  ArrayList time [ms]=53,  LinkedList time [ms]=112,  HashSet time [ms]=22
n=10000000, ArrayList time [ms]=439, LinkedList time [ms]=1023, HashSet time [ms]=219
```

Java Collections – Notes on Performance

- **Note**: The results will (*strongly*) depend on:
  - What operations you perform in the loops
  - What hardware you run on
  - → Take these timings with a grain of salt, not as universal truth!
- **Exercise**: Play around with
  - Different operations in the loop
  - Different ways of iterating (`for` loop, `forEach` loop, use an `Iterator`)
  - Get a sense of what you're dealing with!

Java Collection Use - Examples

- Let's go through some example problems
- Using only the 4 collection types we have seen,
  - `List`
  - `Set`
  - `Map`
  - `Queue`

Which collection type should you use to store the data efficiently?

Example – Email Provider

- Imagine writing software that provides users with accounts to send and receive emails. We need to store:
- **Usernames**
- **Email addresses**
- **Passwords**
- **Emails**

Example – Email Provider

- Imagine writing software that provides users with accounts to send and receive emails. We need to store:
- **Usernames** (unique values → `Set`)
- **Email addresses** (unique values → `Set`)
- **Passwords** (username-password pair → `Map`)
- **Emails** (ordered list → `List`)

Example – Student Database

- Imagine a **student grading system** where we need to store:
- **Student IDs**
- **Student Scores**
- **Student Names**

Example – Student Database

- Imagine a **student grading system** where we need to store:
- **Student IDs** (unique values → `Set`)
- **Student Scores** (name-score pair → `Map`)
- **Student Names** (ordered list → `List`)

Best Practices When Using Collections

✅ **Choose the right collection**

- List for **ordered data**
- Set for **unique values**
- Map for **key-value storage**
- Queue for **task processing**

✅ **Avoid unnecessary resizing**

- Use `ArrayList(int initialCapacity)` to pre-allocate space.
- Use `LinkedList` when frequent insertions/deletions are needed.

Best Practices When Using Collections

✅ Prefer `Iterators` for safe traversal

```
List<Integer> studentIDs = new ArrayList<Integer>();

// do something with it...

Iterator<Integer> it = studentIDs.iterator();
while (it.hasNext()) {
  System.out.println(it.next());
}
```

Best Practices When Using Collections

- ☑ **Use built-in methods for sorting* & searching**

  *(*unless you're really really sure you know what you're doing)*

```
List<String> studentNames = new ArrayList<String>();

// do something with the list...

studentNames.sort();
if (studentNames.contains("Peter")) {
  System.out.println("Peter is here");
}
```

Best Practices When Using Collections

✅ Even better: **Use `Collections` Utility Class**

Benefit: Applicable to all Collections! `Collections` class is agnostic of type of `Collection (List, Set, Map...)` you use.

```
import java.util.Collections;

List<Integer> studentIDs =
                    new ArrayList<Integer>();

// do something with the list...

Collections.sort(studentIDs);
int max = Collections.max(studentIDs);
int min = Collections.min(studentIDs);
```

Useful methods of `Collections`:

- **`sort()`** – sort.
- **`shuffle()`** – randomly shuffle elements
- **`fill()`** – fill collection with some value
- **`reverse()`** – reverse order
- **`swap()`** – swap two elements
- **`min()`** – get min
- **`max()`** – get max

Durham University

Durham University

Some Pitfalls With Collections

Objects vs References in Java

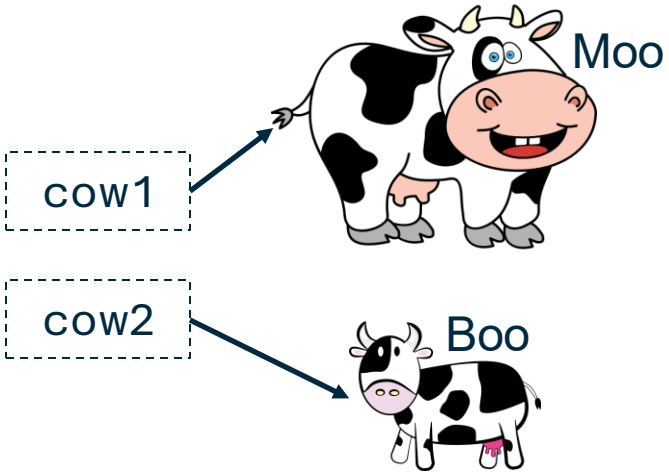
Understanding the difference between **objects** and **references** in Java is fundamental for mastering memory management, object-oriented programming, and debugging common issues in Java applications.

Recap: Objects vs References Overview

- A **reference** in Java is a variable that stores the memory address of an object.
- An **object** is an instance of a class that resides in memory (heap).
- A reference does not hold the actual object; instead, it points to it.

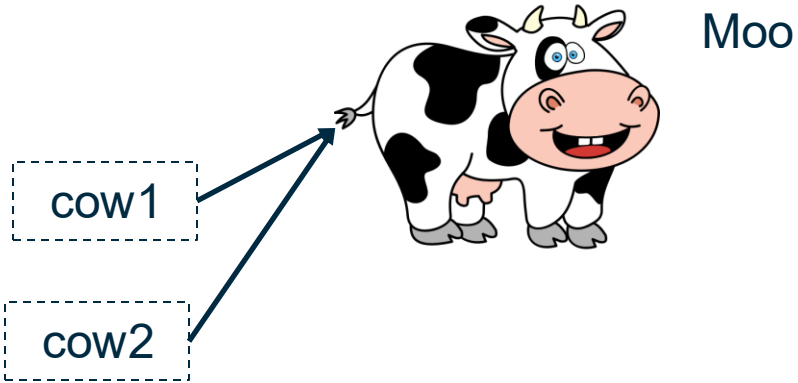
Recap: Visualising Object Creation

```
Cow cow1 = new Cow("Moo", 40.0f);
Cow cow2 = new Cow("Boo", 20.0f);
```



Recap: Visualising Object Creation

```
Cow cow1 = new Cow("Moo", 40.0f);
Cow cow2 = cow1;
```



`null` References

- If a reference doesn't point to any object, it holds **`null`**.
- **Example of `NullPointerException`:**

```
Car c1 = null;
c1.model = "BMW"; // causes NullPointerException
```

- Always check for **`null`** before accessing an object!

```
if (c1 == null) {
  System.out.println("Error!");
}
```

Durham University

Stack vs Heap

- In Java (and C, C++) There are 2 "ways"/"regions"/"areas" where memory is allocated:
- **Stack**:
  - For local variables and function calls
- **Heap:**
  - For dynamically allocated memory during program execution

Stack

- Whenever a function is called, the required memory for its **local variables** is already **known beforehand**.
- This memory gets allocated on the "**stack**".
  - When a function calls another function internally, this second function's variables are ***stacked*** on top
- Once function completes, its memory is **automatically freed**
  - *Removed from the top of the stack*
  - The developer doesn't need to do anything.

Heap

- The heap stores memory allocated **dynamically** during runtime
  - = for everything else.
  - "just throw it on the ~~pile~~ heap with the rest of it"

- **Not automatically** freed:
  - Needs manual deallocation (C, C++)
  - … or a **garbage collector** (Java, Python, …)

Garbage Collection & Unreachable Objects

- If an object has no reference, it becomes **eligible for garbage collection**.
- Example:

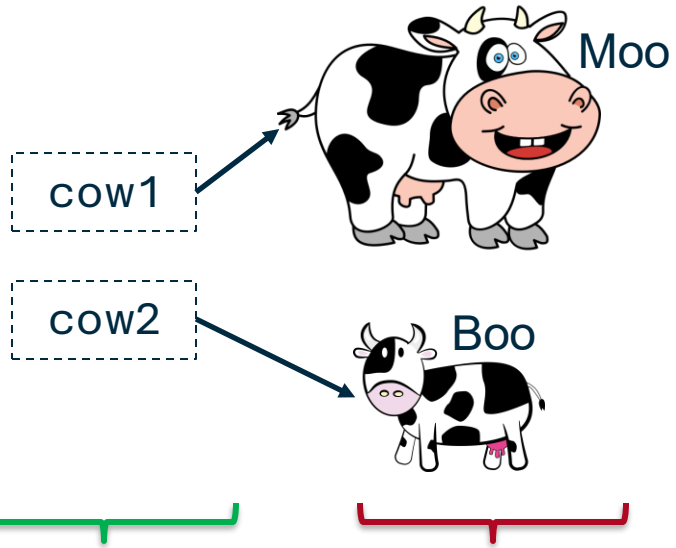
```
Cow c1 = new Cow();
c1 = new Cow(); // first object is now unreachable
```

- Java **automatically** reclaims memory.
  - In other languages, such as C/C++, you must do that manually.

Recap: Visualising Object Creation

```
Cow cow1 = new Cow("Moo", 40.0f);
Cow cow2 = new Cow("Boo", 20.0f);
```

- Here, "**data of local variables**" is reference (address) of objects.



Durham University

Objects vs References: A Different Example

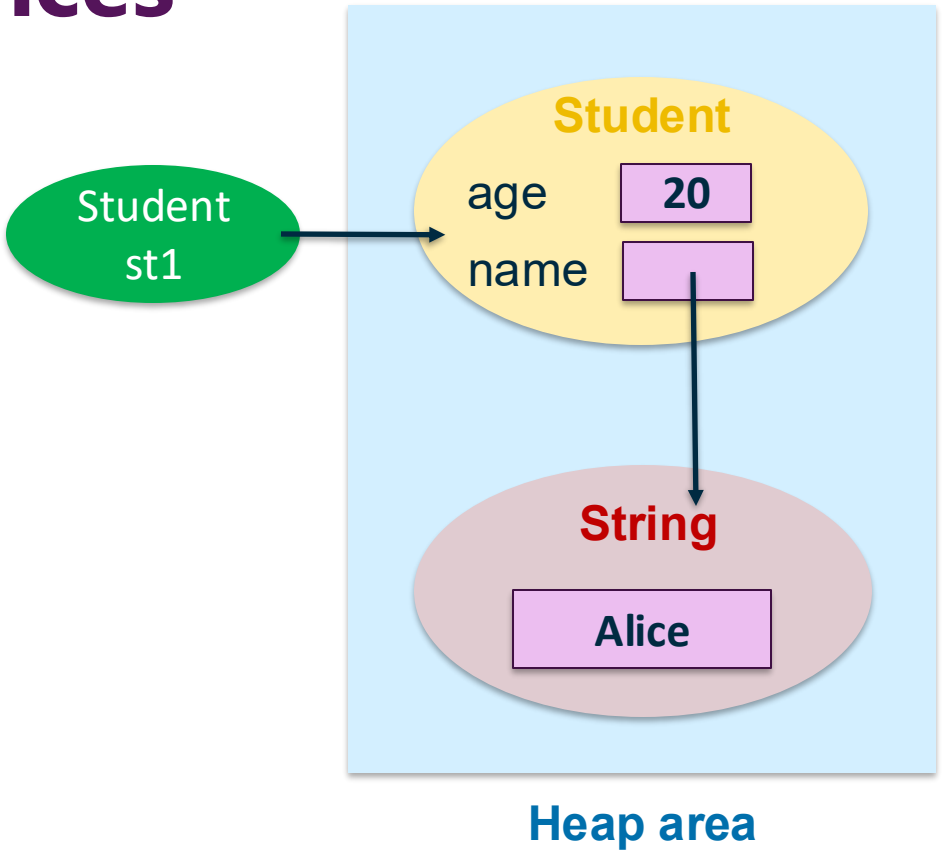
- The objects are created in the **heap area**
- the references `st1`, `st2` just point out to the object of the `Student` class in the heap
- **But**: Our class contains objects itself, what happens to that data?

```
public class Student {
  String name = "Alice";
  int age = 20;
  void sayHello() {
    System.out.println("Hello");
  }
}

public class RunStudent {
  public static void main(String[] args) {
    Student st1 = new Student();
    Student st2 = new Student();
  }
}
```

Objects vs References

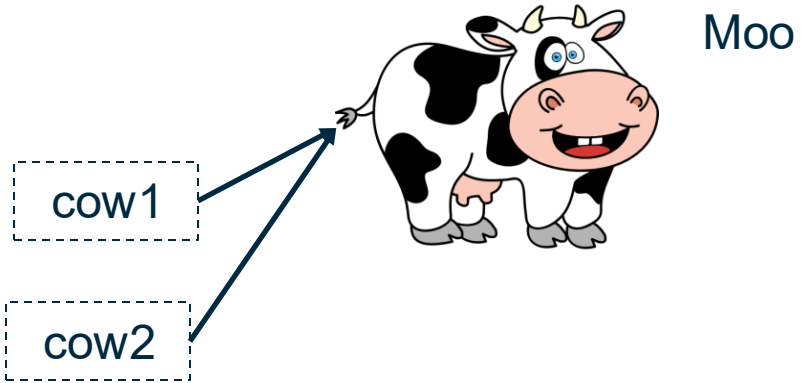
- since the `String` is also an object, the `Student` object's `name` variable is also a reference that points to the actual `String` value (`"Alice"`).



Objects vs References: Multiple Variables

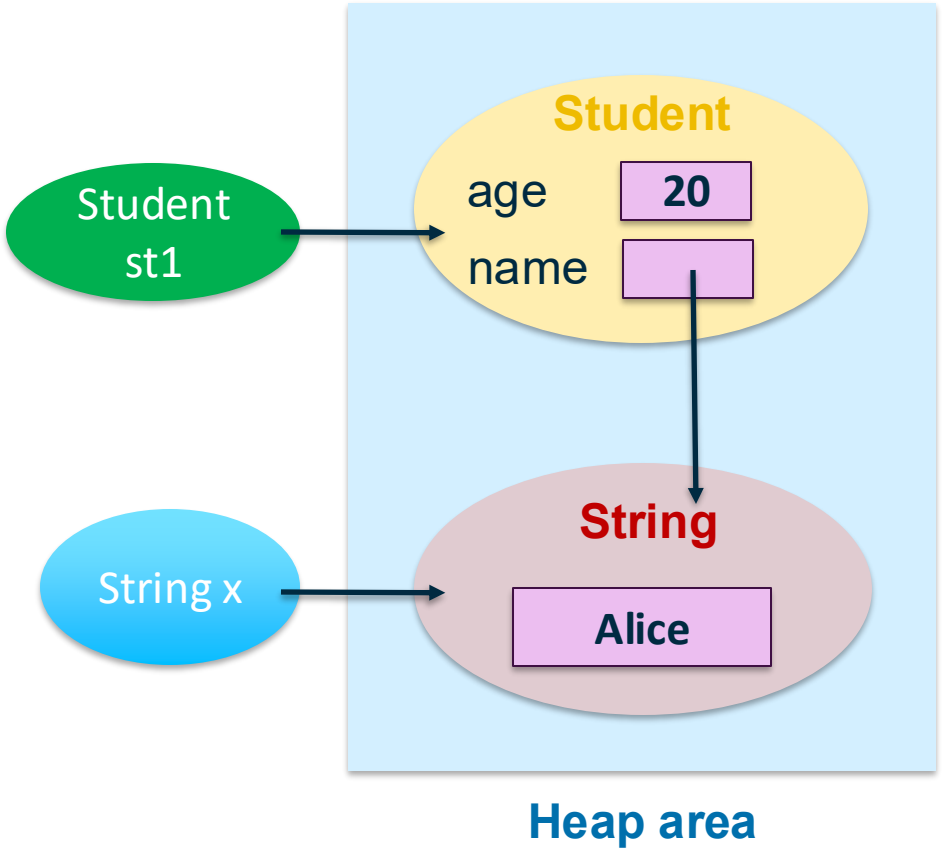
But remember: We can have **multiple variables** pointing to **the same object**!

```
Cow cow1 = new Cow("Moo", 40.0f);
Cow cow2 = cow1;
```



Objects vs References: Multiple Variables

… so we could have something like this, too!



Understanding Objects and References in Java Collections

- We store **`Person`** objects in an `ArrayList`.
- After storing the object, we modify it.
- Result: The `Person` object in the `ArrayList` has changed as well!

```java
import java.util.ArrayList;

class Person {
    String name;
    public Person(String name) {
        this.name = name;
    }
}

public class ReferenceExample {
    public static void main(String[] args) {
        ArrayList<Person> people = new ArrayList<>();

        Person p1 = new Person("Alice");
        people.add(p1);

        p1.name = "Bob"; // Modifies the object inside the ArrayList
        System.out.println(people.get(0).name); // Output: Bob
    }
}
```

Durham University

Understanding Objects and References in Java Collections

What went wrong?

- `p1` is a **reference**, and modifying `p1.name` changes the object's data on the heap.
- `ArrayList` also only contains a reference to the object's data on the heap, not the object's data.
- **Misconception:** Some assume modifying `p1` does not affect the collection.

```java
import java.util.ArrayList;

class Person {
    String name;
    public Person(String name) {
        this.name = name;
    }
}

public class ReferenceExample {
    public static void main(String[] args) {
        ArrayList<Person> people = new ArrayList<>();

        Person p1 = new Person("Alice");
        people.add(p1);

        p1.name = "Bob"; // Modifies the object inside the ArrayList
        System.out.println(people.get(0).name); // Output: Bob
    }
}
```

Cloning Objects to Avoid Shared References

- **Example:** Preventing shared references using a **Copy Constructor**
- **Copy Constructor:**
  - Takes **object of same class as argument**
  - Specify **how to copy** over data from argument to this object
  - There is **no default** copy constructor, you always have to write your own. Java won't do it for you as it does for regular constructors!

```java
class Person {
    String name;
    public Person(String name) {
        this.name = name;
    }
    public Person(Person other) { // Copy Constructor
        this.name = other.name;
    }
}

public class CloneExample {
    public static void main(String[] args) {
        ArrayList<Person> people = new ArrayList<>();

        Person p1 = new Person("Alice");
        people.add(new Person(p1)); // Stores a copy

        p1.name = "Bob";
        System.out.println(people.get(0).name); // Output: Alice
    }
}
```

Cloning Objects to Avoid Shared References

- **Example:** Preventing shared references using a **Copy Constructor**
- **Fix Explanation**:
  - Instead of storing the original reference, we store a new copy of `Person`.
  - Now, modifying `p1.name` does not affect the collection.

```java
class Person {
    String name;
    public Person(String name) {
        this.name = name;
    }
    public Person(Person other) { // Copy Constructor
        this.name = other.name;
    }
}

public class CloneExample {
    public static void main(String[] args) {
        ArrayList<Person> people = new ArrayList<>();

        Person p1 = new Person("Alice");
        people.add(new Person(p1)); // Stores a copy

        p1.name = "Bob";
        System.out.println(people.get(0).name); // Output: Alice
    }
}
```

Summary

- **Examples using Java Collections**
- **Using Objects and References**

Durham University

The End

Any Questions?